

# 版本/分支感知的多状态记忆闭环

TencentDB Agent Memory 实现原理与真实评测 | 2026-08-31 | 分支 codex/version-aware-multistate-memory | 评分前提交 4d77e69

技术结论：版本能力 criterion accuracy 为 100.00%：对全局混合召回 +5.71 点 [2.86, 8.57]，对线性新旧双态 +32.50 点 [30.36, 34.64]。期望状态精确率 0% → 100%，污染率 100% → 0%，能力面板注入 token 分别减少 29.04% 和 18.80%。

## 1 核心原理：同域更新，跨域并存

线性“旧—新”关系默认两个事实描述同一个世界状态。但 main 与 release、两个 worktree、同一 worktree 中两个并行任务可能互相矛盾却同时正确。新方案把线性双态保留为一个特例：只有有效域完全相同，后写事实才可能更新前序事实；有效域不同，状态并列保存，读取时再按执行环境和查询意图选择。



### 状态坐标与有效性

每条 L1 记忆携带  $M = (\text{content}, \text{repositoryId}, \text{branch}, \text{commitSha}, \text{worktreeId}, \text{taskId}, \text{scopeLevel}, \text{provenance})$ 。repo 和 worktree 是不可逆短哈希，不落远端 URL 或本机路径；commit 用于溯源。detached worktree 还必须匹配提交，避免不同 HEAD 状态串用。

| 层级         | 有效域键                      | 适用条件         | 典型内容             |
|------------|---------------------------|--------------|------------------|
| repository | repo                      | 仓库相同         | 项目级构建约定、目录结构     |
| branch     | repo + branch             | 再匹配分支        | release 分支测试命令   |
| worktree   | repo + branch + wt        | 再匹配 worktree | 未提交配置、实验环境       |
| task       | repo + branch + wt + task | 再匹配 taskId   | 并行 Agent 假设、临时命令 |

### 四条不变量

① 同仓库不等于同状态：兄弟分支、worktree、task 不能互相更新。② 写入域与读取域一致：去重、合并、删除、纠错都使用精确域键。③ 越具体越优先：普通查询按 task → worktree → branch → repository。④ 无上下文则弃权：只保留 legacy unscoped，抑制所有 scoped 记忆。

## 2 写入闭环：坐标随 L0 一直延续到 L1

### Git 与 worktree 坐标

OpenClaw 或 Gateway 在 capture 时传 workspaceDir、稳定 taskId，或直接传显式 versionContext。Git 探测器读取 common-dir、当前分支、HEAD、worktree 顶层和可选远端：有远端时用规范化远端身份生成 repositoryId，无远端时使用 common-dir；worktree 顶层生成不同 worktreeId；detached HEAD 记为 detached@commit。探测缓存 2 秒，单条 Git 命令默认 250 ms 超时；失败时返回无上下文，不猜测身份。

### 并行任务不会在异步管线中串域

capture 先写 L0，原有 team、user、agent、session、task 隔离保持不变。版本坐标按 sessionKey + sessionId + taskId 的哈希键持久化，异步 L0→L1 再加载同一坐标。把 taskId 纳入键，避免同一会话中后到任务覆盖先到任务的版本上下文。

### 精确域内才允许更新

L1 去重和写入先执行租户隔离，再比较 memoryVersionWriteDomainsEqual。branch 键为 repo + branch；worktree 再加入 wt；task 再加入 taskId。只有精确域相同的旧记录才可能去重、update/merge、删除或建立纠错边。原始证据保留；新记录可查询、旧 ID 全部核验且置信度达标后，读取侧才解析同域前驱—后继。跨域记录不会进入旧 ID 集合。

## 3 召回闭环：先检索，再确定版本视图

最终路径是 performAutoRecall。MemoryCore 先通过 SQLite/FTS5、embedding 或 hybrid/RRF 检索；严格模式按 versionCandidateMultiplier 过取候选，本实验为最终 Top-k 的 4 倍，再经过生命周期解析和版本选择。这样可避免正确状态因其他分支占满原始 Top-k 而无法进入作用域选择器。

| 查询模式        | 候选范围            | 选择与标签                                           |
|-------------|-----------------|-------------------------------------------------|
| 当前执行        | 当前仓库适用状态        | task → worktree → branch → repo；标记 ACTIVE SCOPE |
| 比较/迁移/回归/历史 | 同仓库多个状态         | 每个域先保留一条，再补齐；标记 VERSION STATE                   |
| 点名两个分支      | 被点名分支           | 有 branch 状态时不误带其下全部 task/worktree               |
| 缺少仓库上下文     | legacy unscoped | scoped 全部弃权，不猜当前状态                              |

## 4 TencentDB Agent Memory 接入

多状态返回同时受 maxVersionStates、最终 resultLimit 和提示 token 预算约束；标签包含 branch、commit、worktree、task、scope 与 active\_here。决策日志记录意图、输入候选、抑制数、标签数、当前态数、fallback 和延迟。Reader/Judge 或人工反馈用于调整选择策略，不直接触发跨域事实改写。

| 环节     | 代码位置                                           | 职责                                |
|--------|------------------------------------------------|-----------------------------------|
| 坐标与选择  | lifecycle/version-scope.ts                     | 域键、有效性、意图、标签和有界多状态                |
| Git 探测 | lifecycle/git-context.ts                       | 仓库/worktree 身份、detached HEAD、隐私哈希 |
| 异步延续   | version-context-store.ts · pipeline-factory.ts | 按会话和 task 保存坐标，防止并行串域             |
| 写入去重   | record/l1-dedup.ts · l1-writer.ts              | 同域更新，兄弟状态并存                       |
| 最终召回   | hooks/auto-recall.ts                           | 候选过取 → 生命周期 → 版本筛选 → 提示           |
| 宿主入口   | tdai-core.ts · index.ts · gateway/*            | OpenClaw / Gateway 传工作区、task、显式坐标 |

### 参考配置

```
{ "recall": { "lifecycle": { "enabled": true, "versionAwareMode": "strict",
  "autoDetectGit": true, "versionCandidateMultiplier": 4, "maxVersionStates": 6 } } }
```

自动探测默认采用 worktree 级作用域；并行任务必须传稳定 taskId。确需跨 worktree 共享的分支事实，由宿主显式传 scopeLevel=branch。

## 5 测试数据集

### 版本能力面板：70 个受控生成题

10 个编程状态场景 × 7 类问题，共 70 题。状态文本由实验生成，所以不是自然分布数据；但所有状态都落入真实 Git、真实 SQLite 和 Agent Memory 最终召回路径。实验仓库包含 main、release、两个 detached worktree 和 parallel-alpha / parallel-beta；worktree 状态还写入真实文件。

| 切片                       | 题数 | 期望行为                  |
|--------------------------|----|-----------------------|
| branch current           | 10 | 只返回当前分支事实             |
| worktree current         | 10 | 同分支只返回当前 worktree     |
| parallel task current    | 10 | 同 worktree 只返回当前 task |
| branch comparison        | 10 | 返回两个指定分支并标注           |
| migration                | 10 | 返回迁移两端，不混入无关 task     |
| regression               | 10 | 返回定位变化所需状态            |
| missing scope abstention | 10 | 无仓库坐标时不选 scoped 当前值   |

### Memora 公开安全面板：40 题

公开数据固定在提交 a6493188efc836d6511ed5e4163fe3ba87da30ff: 30 group、10 persona、600 问题、27,614 session、24,856 memory unit，覆盖 activity/preference/goal 的 add、update、delete 与 no-memory。本实验从先前冻结的 weekly 自然题中，对 10 个 persona 各按字典序取前 4 题，共 40 题。选择发生在读取新答案和 verdict 前。

这 40 题只验证 legacy unscoped 路径不退化。三组提示逐字节相同；它是复用的公开安全面板，不是新的外部留出集，也不证明版本作用域能力。

6 实验方法：真实生产路径加交叉答案评测

| 方案      | 版本处理                   | 用途         |
|---------|------------------------|------------|
| 全局混合    | 不做版本过滤；Top-k 状态都可直接使用  | 强答案基线与污染对照 |
| 线性新旧双态  | 并行状态故意串成 old→current 链 | 复现双态语义限制   |
| 版本感知多状态 | 当前态过滤；比较类带标签多状态；缺上下文弃权 | 被测方案       |

生产路径与完整性

| 层级    | 真实执行                                      | 规模 / 完整性                      |
|-------|-------------------------------------------|-------------------------------|
| Git   | main、release、两个 detached worktree、两个 task | 同 repold、不同 worktreeId；路径不持久化 |
| 存储    | 独立 SQLite/FTS5 + writeMemory              | 50 DB；260 次写入                 |
| 最终召回  | performAutoRecall + 生命周期 + 版本选择 + 提示预算    | 330 次；fallback 0              |
| 上下文校验 | 当前态、比较集、缺失 scope、自然提示等                    | 全部通过；四类 mismatch 均 0          |
| 答案链路  | 双 Reader，答案仅由另一模型 Judge                   | 500 Reader + 500 Judge；无自评    |

Reader/Judge 与评分

110 题 × 3 组得到 330 个 case-arm 上下文。40 个自然题只复用同一 case 内逐字节相同的消息，因此每个 Reader 为 250 个唯一提示。Reader 为 MiniMax-M3（temperature 0.1、top-p 0.95）与 deepseek-v4-flash（temperature 0），thinking 均关闭；每个答案只由另一个固定模型逐 criterion 判断。共 500 Reader + 500 Judge，重试、模型错配、自评和 unclear verdict 均为 0。

| 指标                 | 定义                                              | 角色          |
|--------------------|-------------------------------------------------|-------------|
| criterion accuracy | 正确 criterion / 全部 criterion                     | 70 题能力主指标   |
| MPA                | presence criterion 正确比例                         | 应出现记忆是否被使用  |
| FAA                | absence criterion 正确比例                          | 错误状态是否被避开   |
| FAMA               | $\max(0, \text{MPA} - \lambda(1 - \text{FAA}))$ | 同时惩罚遗漏与错误保留 |
| 精确选择率              | 实际 ID 集合 = 期望集合                                 | 回答前的集合正确性   |
| 状态召回率              | 命中期望状态 / 全部期望状态                                 | 必要状态完整性     |
| 污染率                | 含任一非期望状态的 case 比例                               | 跨域污染        |
| token/条目/延迟        | 最终提示 token、ID 数、本地耗时                            | 上下文与工程成本    |

主比较使用 5,000 次配对聚类 bootstrap：能力题按 10 个场景聚类，安全题按 10 个 persona 聚类，报告 95% 区间。评分前代码固定为 4d77e698...；上下文 SHA-256 为 2c64c12d...a1960f。

7 结果：上下文状态从混合变为精确

| 70 题能力面板           | 全局混合     | 线性新旧双态   | 版本感知多状态  |
|--------------------|----------|----------|----------|
| criterion accuracy | 94.29%   | 67.50%   | 100.00%  |
| MPA                | 94.29%   | 47.14%   | 100.00%  |
| FAA                | 94.29%   | 87.86%   | 100.00%  |
| FAMA               | 94.29%   | 47.14%   | 100.00%  |
| 期望集合精确率            | 0.00%    | 0.00%    | 100.00%  |
| 期望状态召回率            | 85.71%   | 21.43%   | 100.00%  |
| 跨状态污染率             | 100.00%  | 100.00%  | 0.00%    |
| 平均注入 token         | 557.89   | 487.57   | 395.90   |
| 平均召回条目             | 4.00     | 2.03     | 1.29     |
| 本地平均召回             | 0.622 ms | 0.822 ms | 0.569 ms |

criterion accuracy 相对全局混合 +5.71 点 [2.86, 8.57]，相对线性双态 +32.50 点 [30.36, 34.64]。能力面板 token 分别 -29.04% / -18.80%，条目 -67.86% / -36.62%。本地耗时分别 -8.52% / -30.78%，但不代表远端 TCVDB/COS 延迟。

各能力点

| 能力点          | 全局混合    | 线性双态    | 新方案     | 提升（对前两组）          |
|--------------|---------|---------|---------|-------------------|
| 分支当前态        | 100.00% | 42.50%  | 100.00% | +0.00 / +57.50 点  |
| worktree 当前态 | 100.00% | 50.00%  | 100.00% | +0.00 / +50.00 点  |
| 并行任务当前态      | 100.00% | 50.00%  | 100.00% | +0.00 / +50.00 点  |
| 分支比较         | 100.00% | 100.00% | 100.00% | +0.00 / +0.00 点   |
| 迁移           | 100.00% | 100.00% | 100.00% | +0.00 / +0.00 点   |
| 回归           | 100.00% | 100.00% | 100.00% | +0.00 / +0.00 点   |
| 缺失 scope 弃权  | 60.00%  | 30.00%  | 100.00% | +40.00 / +70.00 点 |

对全局混合：7 题改善、63 持平、0 受损，两位 Reader 方向 +2.86 / +8.57 点。对线性双态：40 题改善、30 持平、0 受损，两位 Reader +27.86 / +37.14 点。比较、迁移、回归答案同为满分，但新方案只保留所需分支状态，不把无关 worktree/task 交给模型。

8 结果解释、实际优势与证据边界

为什么对全局混合的答案只提升 5.71 点？

受控记忆正文保留 State owner，两种强 Reader 即使看到 4 个混合状态，也能在多数简单题中自行消歧，所以分支、worktree、任务当前态及跨分支问题的答案分数持平。然而全局基线的期望集合精确率仍为 0%、污染率为 100%。新方案的关键变化发生在回答之前：由系统验证有效域，减少无关状态与 token，而不是依赖模型从混合上下文中猜测。

Memora 安全结果

40 个公开问题的三组提示和答案完全相同：criterion accuracy 均为 8.49%，FAMA 均为 2.19%，差值 0.00 点，40 题全部持平。绝对值较低，说明冻结的少量召回上下文本身不足；这里仅证明 legacy unscoped 路径不退化，不能宣称通用记忆质量提升。

实际使用优势

| 场景         | 原问题                   | 多状态机制的作用                 |
|------------|-----------------------|--------------------------|
| 长期维护分支     | release 被 main 最新写入覆盖 | 两个分支同时有效；普通执行只取当前分支      |
| 多 worktree | 同分支本地实验相互污染           | worktree 哈希隔离；比较时才联合     |
| 并行 Agent   | 同会话最后写入覆盖其他任务         | session + taskId 延续各自上下文 |
| 迁移与回归      | 只保留最新会失去对照            | 返回有界、带来源、可追溯状态           |
| 上下文缺失      | 任意选择一个分支值             | scoped 记忆弃权，避免无依据猜测      |
| 提示预算       | 模型阅读所有相关但无效状态         | 确定性过滤后才占用 token          |

证据边界

| 已按冻结协议证明                                                                                      | 尚未覆盖                                                                                  |
|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 真实 Git/worktree、隐私坐标、并行 task 延续、SQLite 写入、同域去重、最终召回、版本选择、标签、双 Reader/交叉 Judge、legacy 兼容与独立复算。 | 自动 LLM 抽取 scopeLevel；在线 TCVDDB/COS 延迟；复杂 merge/cherry-pick/分支重命名；自然流量冲突率；人类 Judge 标定。 |

证据支持的判断：在冻结的版本能力面板和真实 Agent Memory 生产召回路径内，版本作用域机制有效；它把线性双态扩展为可并存的多状态，并在不改变旧式无作用域路径的前提下消除了本实验中的跨状态污染。该结论不外推到尚未测试的自动 scope 抽取、远端存储延迟和无控制自然流量。

独立验证 500 条原始 verdict 全量重算 | status = passed | mismatchCount = 0

审计哈希 上下文 2c64c12d...a1960f | 原始评测 e546f4f9...65ee61 | 汇总 fbe7d455...e61 | 复算 758cade8...678

证据目录 results/version-aware-final/context/context-manifest.json · e2e/evaluations.jsonl · e2e/summary.json · e2e/independent-validation.json